TEAM NAME

TimeGuard AI

Predicting machine failure from sensor streams

A maintenance window with uncertainty and a reason to act

[Insert registered team name]

Stage 1 · 19 October 2026

Team members

[Member 1]	RUL modeling and validation
[Member 2]	Data pipeline and stream replay
[Member 3]	Interface and visual explanation
[Member 4]	Integration, evidence and pitch

Selected problem statement

Track 1

AI/ML Methodology

Challenge 10: TimeGuard AI

Predicting Machine Failure from Sensor Streams

Difficulty: Intermediate to Advanced

Problem understanding

Late warnings are costly

Overestimated remaining life delays intervention until an avoidable outage. A low average error can hide expensive late predictions.

Repeated alerts erode trust

Noisy sensors and short spikes create alert fatigue. Operators need persistent evidence and a useful maintenance horizon.

Drift, missing packets and operating changes can invalidate a confident forecast.

Proposed solution

A range for maintenance planning

Forecast remaining useful life, show a prediction interval, and require persistent threshold crossings before escalating a maintenance review.

Evidence for an operator

Explain sensor contributions, preserve telemetry freshness, and request review when the input leaves its validated envelope.

Live replay, perturbation controls and export make the concept demonstrable with zero hardware.

Innovation / novelty

Asymmetric decisions	A lower decision quantile reflects a higher cost for late action.
Measured uncertainty	Conformal calibration reports coverage together with interval width.
Actionable evidence	Local model attribution connects the forecast to observed sensor trends.
Quality-aware alerts	Persistence, hysteresis and review states control alert fatigue.

Proposed technology / AI approach

C-MAPSS: sequential RUL

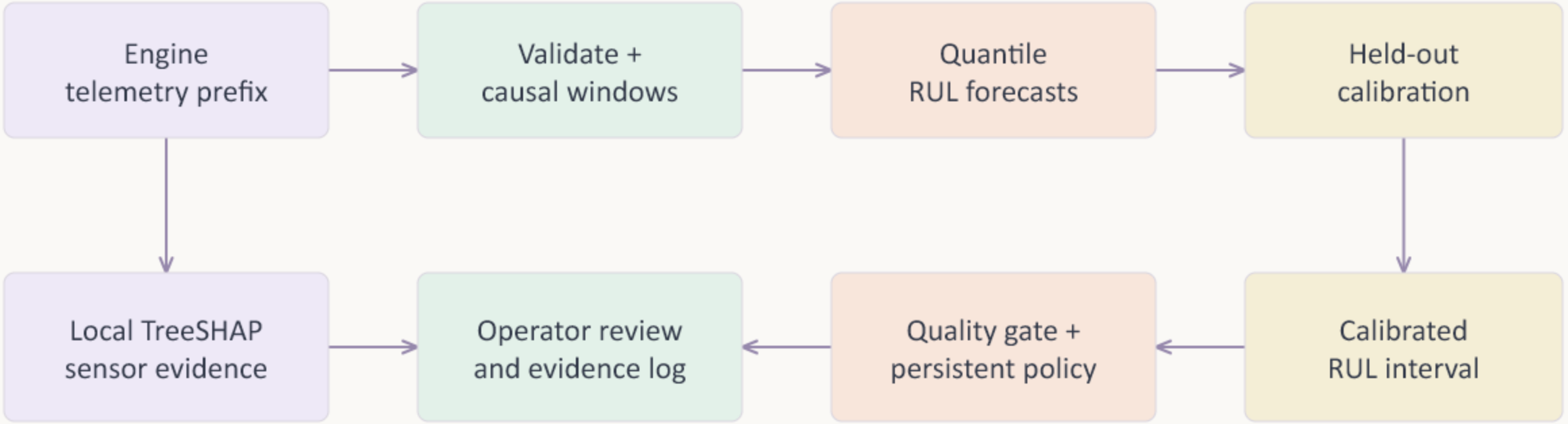
30-cycle causal EMA windows
Trailing moments and slopes
Boosted 5th / 50th / 95th quantiles
Held-out engine calibration

AI4I: separate diagnosis

Physical process and type inputs
Binary failure + five mode labels
Balanced random forests
No IDs or target labels as inputs

Decision quantile τ = 0.20 for an illustrative 4:1 late-to-early cost ratio. RUL training cap: 125 cycles.

Technical approach flowchart



Separate AI4I branch: manufacturing snapshot → binary failure + multilabel modes → diagnosis workbench

Expected impact

20%

Target reduction in simulated decision cost versus a tuned baseline

90%

Nominal interval coverage with measured width and assumptions

<100 ms

Target model-only p95 latency on a declared CPU

Illustrative gross value: ₹20 lakh/year

10 avoided outages × 4 hours × ₹50,000/hour. Subtract maintenance and deployment costs.

Brief implementation plan

00:00–01:30	Data validation, causal features and baseline
01:30–03:15	RUL quantiles, conformal calibration and AI4I models
03:15–04:45	Replay, maintenance policy and local explanations
04:45–06:00	Locked evaluation, ablations and stable core demo
06:00–07:30	Rehearsal, results, README and final submission

Demonstration and evidence

Working Stage 1 portal

Pastel interface and deterministic replay
Pause, reset, step and speed controls
Sensor offset and packet-loss scenarios
Decision log and CSV export

Stage 2 evidence to publish

Endpoint RMSE and NASA S-score
Coverage, width and late-estimate rate
Per-mode precision, recall and PR-AUC
Latency and simulated decision cost

Portal values are synthetic. Repository reports contain measured benchmark results. Industrial ROI remains unvalidated.

Initial benchmark results

RUL and uncertainty

Median raw RMSE: 14.90 cycles
Late estimates: 65% median / 28% decision
Nominal 90% interval: 84% test coverage
Mean interval width: 54.27 cycles

Classification and latency

AI4I precision: 94.92%
AI4I recall: 82.35%
AI4I F1: 88.19%
Four-model RUL p95: 7.92 ms

Limits: baseline RUL accuracy is better. Coverage misses target. TWF and RNF recall are zero.